

**LAPORAN PRAKTIKUM**  
**MATERI 2 (TUGAS 1 & 2) :**  
**SISTEM MANAJAMEN PROSES DAN CHAT APPLICATION**  
**DENGAN IPC**



**Dosen Pengampu :**  
**Triawan Adi Cahyanto M.Kom**

**Disusun Oleh :**  
**MUHAMMAD RIDHO 2410651009 (A)**

**Tanggal : 10-20-2025**

**PRODI TEKNIK INFORMATIKA**  
**FAKULTAS TEKNIK**  
**UNIVERSITAS MUHAMMADIYAH JEMBER**  
**2025**

## TUGAS 1

### 1. Tujuan :

Tujuan dari praktikum ini adalah untuk memahami konsep dasar manajemen proses pada sistem operasi, termasuk pembuatan proses anak (child process), pengaturan tugas antar proses, serta pengumpulan hasil dari proses-proses tersebut oleh proses induk (parent process).

### 2. Dasar Teori :

#### a. Konsep Proses

Proses adalah program yang sedang dieksekusi oleh sistem operasi. Setiap proses memiliki *Process ID (PID)*, memori sendiri, dan status eksekusi. Proses dapat membuat proses lain (child process) menggunakan `fork()`.

#### b. IPC (Inter Process Communication)

IPC adalah mekanisme komunikasi antar proses. Metode yang umum digunakan meliputi:

- **Shared Memory** – proses berbagi area memori yang sama.
- **Message Queue** – komunikasi berbasis pesan antrian.
- **Pipe** – saluran komunikasi sederhana antara dua proses.

#### c. Fungsi Sistem Terkait

- `fork()` → membuat proses anak.
- `wait()` → menunggu proses anak selesai.
- `getpid()` dan `getppid()` → mendapatkan PID dan PPID.
- `exec()` → mengganti proses saat ini dengan program lain.
- `clock_gettime()` atau `gettimeofday()` → mengukur waktu eksekusi.

### 3. Ini adalah proses kode source dari awal hingga akhir.

- Analisis dari saya yaitu proses induk membuat satu proses anak menggunakan `fork()`. Setelah proses anak selesai, proses induk menampilkan pesan akhir. Terlihat bahwa kedua proses berjalan bergantian dengan PID berbeda.

```

#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <sys/wait.h>
#include <sys/types.h>
#include <time.h>
#include <string.h>
#include <errno.h>

#define NUM_CHILD 3

long long now_ms(void) {
    struct timespec ts;
    if (clock_gettime(CLOCK_MONOTONIC, &ts) != 0) return 0;
    return (long long)ts.tv_sec * 1000LL + ts.tv_nsec / 1000000LL;
}

int cmp_int(const void *a, const void *b) {
    int ia = *(const int*)a;
    int ib = *(const int*)b;
    return (ia > ib) - (ia < ib);
}

void task_sorting(char *outbuf, size_t bufsz) {
    const int N = 5000;
    int *arr = malloc(sizeof(int) * N);
    if (!arr) {
        snprintf(outbuf, bufsz, "sorting: malloc failed");
        return;
    }
    srand((unsigned int)(getpid() ^ time(NULL)));
    for (int i = 0; i < N; ++i) arr[i] = rand();
    qsort(arr, N, sizeof(int), cmp_int);
    int first = arr[0];
    int median = arr[N/2];
    snprintf(outbuf, bufsz, "sorting: N=%d, first=%d, median=%d", N, first, median);
    free(arr);
}

```

```

int binary_search(int *arr, int n, int key) {
    int l = 0, r = n - 1;
    while (l <= r) {
        int m = l + (r - l) / 2;
        if (arr[m] == key) return m;
        if (arr[m] < key) l = m + 1;
        else r = m - 1;
    }
    return -1;
}

void task_searching(char *outbuf, size_t bufsz) {
    const int N = 20000;
    int *arr = malloc(sizeof(int) * N);
    if (!arr) {
        snprintf(outbuf, bufsz, "searching: malloc failed");
        return;
    }
    for (int i = 0; i < N; ++i) arr[i] = i * 2;
    int key = rand() % (N * 2);
    int idx = binary_search(arr, N, key);
    if (idx >= 0)
        snprintf(outbuf, bufsz, "searching: key=%d found at idx=%d", key, idx);
    else
        snprintf(outbuf, bufsz, "searching: key=%d not found", key);
    free(arr);
}

int is_prime(int x) {
    if (x <= 1) return 0;
    if (x <= 3) return 1;
    if (x % 2 == 0) return 0;
    for (int i = 3; (long long)i * i <= x; i += 2)
        if (x % i == 0) return 0;
    return 1;
}

```

```

void task_calculation(char *outbuf, size_t bufsz) {
    const int LIMIT = 200000;
    long long sum = 0;
    int count = 0;
    for (int i = 2; i <= LIMIT; ++i) {
        if (is_prime(i)) {
            sum += i;
            ++count;
        }
    }
    snprintf(outbuf, bufsz, "calculation: primes<=%d count=%d sum=%lld", LIMIT, count, sum);
}

int main(void) {
    pid_t children[NUM_CHILD];
    int pipes[NUM_CHILD][2];
    long long start_ms_for_index[NUM_CHILD];

    for (int i = 0; i < NUM_CHILD; ++i) {
        if (pipe(pipes[i]) == -1) {
            perror("pipe");
            return 1;
        }
    }
}

```

```

printf("Parent PID=%d. Spawning %d children...\n", (int)getpid(), NUM_CHILD);

for (int i = 0; i < NUM_CHILD; ++i) {
    long long t0 = now_ms();
    pid_t pid = fork();
    if (pid < 0) {
        perror("fork");
        return 1;
    } else if (pid == 0) {
        for (int j = 0; j < NUM_CHILD; ++j) {
            if (j != i) {
                close(pipes[j][0]);
                close(pipes[j][1]);
            }
        }
        close(pipes[i][0]);
    }
}

```

```

    long long child_t0 = now_ms();
    char result_buf[256] = {0};
    srand((unsigned int)(getpid() ^ child_t0));
    if (i == 0) {
        task_sorting(result_buf, sizeof(result_buf));
    } else if (i == 1) {
        task_searching(result_buf, sizeof(result_buf));
    } else if (i == 2) {
        task_calculation(result_buf, sizeof(result_buf));
    } else {
        snprintf(result_buf, sizeof(result_buf), "task %d: noop", i);
    }
    long long child_t1 = now_ms();
    long long duration_ms = child_t1 - child_t0;

    char outmsg[512];
    int written = snprintf(outmsg, sizeof(outmsg), "%d;%lld;%s",
                          (int)getpid(), duration_ms, result_buf);
    if (written < 0) written = 0;
    ssize_t w = write(pipes[i][1], outmsg, (size_t)written);
    if (w == -1) {
        perror("child write");
    }
    close(pipes[i][1]);
    _exit(0);
} else {
    children[i] = pid;
    start_ms_for_index[i] = t0;
    close(pipes[i][1]);
}
}
}

```

```

printf("\nCollecting results...\n");
int remaining = NUM_CHILD;
while (remaining > 0) {
    int status;
    pid_t w = wait(&status);
    if (w == -1) {
        if (errno == ECHILD) break;
        perror("wait");
        break;
    }
    int idx = -1;
    for (int i = 0; i < NUM_CHILD; ++i) if (children[i] == w) idx = i;
    long long end_ms = now_ms();
    long long duration_parent = -1;
    if (idx != -1) duration_parent = end_ms - start_ms_for_index[idx];

    char readbuf[512] = {0};
    if (idx != -1) {
        ssize_t r = read(pipes[idx][0], readbuf, sizeof(readbuf) - 1);
        if (r > 0) readbuf[r] = '\0';
        close(pipes[idx][0]);
    } else {
        strncpy(readbuf, "no pipe data (unknown child)", sizeof(readbuf)-1);
    }
}

```

```

int child_pid = (int)w;
long long child_reported_ms = -1;
char textmsg[400] = {0};
if (strlen(readbuf) > 0) {
    char *p1 = strchr(readbuf, ';');
    if (p1) {
        *p1 = '\0';
        char *p2 = strchr(p1 + 1, ';');
        if (p2) {
            *p2 = '\0';
            child_reported_ms = atoll(p1 + 1);
            strncpy(textmsg, p2 + 1, sizeof(textmsg) - 1);
        } else {
            strncpy(textmsg, p1 + 1, sizeof(textmsg) - 1);
        }
    } else {
        strncpy(textmsg, readbuf, sizeof(textmsg) - 1);
    }
} else {
    strncpy(textmsg, "(no message)", sizeof(textmsg)-1);
}

char status_str[128];
if (WIFEXITED(status)) {
    int exitcode = WEXITSTATUS(status);
    snprintf(status_str, sizeof(status_str), "exited (code %d)", exitcode);
} else if (WIFSIGNALED(status)) {
    int sig = WTERMSIG(status);
    snprintf(status_str, sizeof(status_str), "killed by signal %d", sig);
} else {
    snprintf(status_str, sizeof(status_str), "unknown status");
}

```

```

printf("-----\n");
printf("Child PID: %d\n", child_pid);
if (idx != -1) printf("Task index: %d\n", idx);
if (child_reported_ms >= 0) printf("Execution time (child measured) : %lld ms\n", child_reported_ms);
if (duration_parent >= 0) printf("Execution time (parent measured): %lld ms\n", duration_parent);
printf("Status: %s\n", status_str);
printf("Result: %s\n", textmsg);
printf("-----\n\n");

--remaining;
}

printf("All children collected. Parent exiting.\n");
return 0;
}

```

```

root@RIDHO:~# nano proses_manager.c
root@RIDHO:~# gcc -o proses_manager proses_manager.c
root@RIDHO:~# ./proses_manager
Parent PID=47. Spawning 3 children...

Collecting results...
-----
Child PID: 49
Task index: 1
Execution time (child measured) : 0 ms
Execution time (parent measured): 1 ms
Status: exited (code 0)
Result: searching: key=4749 not found
-----

-----
Child PID: 48
Task index: 0
Execution time (child measured) : 1 ms
Execution time (parent measured): 1 ms
Status: exited (code 0)
Result: sorting: N=5000, first=223935, median=1056335650
-----

-----
Child PID: 50
Task index: 2
Execution time (child measured) : 20 ms
Execution time (parent measured): 21 ms
Status: exited (code 0)
Result: calculation: primes<=200000 count=17984 sum=1709600813
-----

All children collected. Parent exiting.

```

#### 4. Kesimpulan :

- Mahasiswa memahami cara membuat dan mengatur proses menggunakan fork() dan wait().
- Proses dapat berjalan paralel dengan tugas yang berbeda.
- IPC dan manajemen proses penting dalam sistem operasi multitasking.
- Dapat dibandingkan performa antara proses yang melakukan tugas berbeda.

## Tugas 2 : CHAT APPLICATION DENGAN IPC

### 1. Tujuan :

- Memahami cara kerja komunikasi antar proses (Inter-Process Communication / IPC).
- Mengimplementasikan aplikasi chat sederhana dengan menggunakan *Shared Memory* dan *Semaphore*.
- Melatih sinkronisasi data antar proses secara konkuren menggunakan *Mutex* atau *Semaphore*.

### 2. Dasar Teori :

#### a) Konsep Proses

Proses merupakan program yang sedang dieksekusi. Setiap proses memiliki ruang memori, variabel, dan status sendiri. Sistem operasi mengatur proses melalui *Process Control Block (PCB)* untuk menjaga agar eksekusi berjalan secara teratur dan efisien.

#### b) IPC (Inter-Process Communication)

IPC adalah mekanisme yang memungkinkan proses untuk umum digunakan antara lain:

- **Pipes** – komunikasi antar proses yang memiliki hubungan parent-child.
- **Message Queue** – pengiriman pesan berbentuk antrian.
- **Shared Memory** – proses berbagi area memori yang sama untuk bertukar data.
- **Semaphore / Mutex** – digunakan untuk *sinkronisasi* agar tidak terjadi kondisi *race* saat dua proses mengakses data bersama.

3. Dibawah ini Adalah proses kode source dari awal hingga akhir yang pada akhirnya menghasilkan output chat application antar user.

- **Analisis hasil :**

Program berhasil menunjukkan komunikasi antar proses menggunakan Shared Memory dan Semaphore.

- *Shared Memory* digunakan untuk menyimpan pesan.
- *Semaphore* berfungsi sebagai pengunci agar dua proses tidak menulis pada memori secara bersamaan.
- Setiap proses (User1, User2) dapat menambah dan membaca data secara sinkron.

- **Analisis performa :**

- Waktu respon cepat karena IPC berbasis memori.
- Tidak ada bottleneck karena semua proses berjalan mandiri.
- Semaphore memastikan tidak terjadi *race condition* antar proses.

```
#define _POSIX_C_SOURCE 200809L
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <semaphore.h>
#include <fcntl.h>
#include <sys/mman.h>
#include <unistd.h>
#include <time.h>
#include <errno.h>

#define SHM_NAME "/chat_shared_memory"
#define SEM_MUTEX_NAME "/chat_mutex"
#define MAX_MSGS 10
#define MAX_USER 32
#define MAX_TEXT 256

typedef struct {
    time_t ts;
    char user[MAX_USER];
    char text[MAX_TEXT];
} Message;

typedef struct {
    Message msgs[MAX_MSGS];
    int head;
    int count;
} SharedData;

static SharedData *shared = NULL;
static sem_t *mutex = NULL;
```

```

        shm_fd = shm_open(SHM_NAME, O_RDWR, 0666);
    } else {
        perror("shm_open");
        exit(1);
    }
} else {
    *is_creator = 1;
    ftruncate(shm_fd, sizeof(SharedData));
}

void *addr = mmap(NULL, sizeof(SharedData), PROT_READ | PROT_WRITE, MAP_SHARED, shm_fd, 0);
if (addr == MAP_FAILED) {
    perror("mmap");
    exit(1);
}
close(shm_fd);

shared = (SharedData *)addr;

mutex = sem_open(SEM_MUTEX_NAME, O_CREAT, 0666, 1);
if (mutex == SEM_FAILED) {
    perror("sem_open");
    exit(1);
}

if (*is_creator) {
    sem_wait(mutex);
    shared->head = 0;
    shared->count = 0;
    sem_post(mutex);
}

}

void kirim_pesan(const char *user, const char *text) {
    if (sem_wait(mutex) == -1) {
        perror("sem_wait");
        return;
    }
}

```

```

    }

    int idx;
    if (shared->count < MAX_MSGS) {
        idx = (shared->head + shared->count) % MAX_MSGS;
        shared->count++;
    } else {
        idx = shared->head;
        shared->head = (shared->head + 1) % MAX_MSGS;
    }

    shared->msgs[idx].ts = time(NULL);
    strncpy(shared->msgs[idx].user, user, MAX_USER - 1);
    shared->msgs[idx].user[MAX_USER - 1] = '\0';
    strncpy(shared->msgs[idx].text, text, MAX_TEXT - 1);
    shared->msgs[idx].text[MAX_TEXT - 1] = '\0';

    if (sem_post(mutex) == -1) perror("sem_post");
}

void lihat_history() {
    if (sem_wait(mutex) == -1) {
        perror("sem_wait");
        return;
    }

    if (shared->count == 0) {
        printf("Belum ada pesan.\n");
        sem_post(mutex);
        return;
    }

    for (int i = 0; i < shared->count; ++i) {
        int idx = (shared->head + i) % MAX_MSGS;
        struct tm tm;
        char timestr[32];
        localtime_r(&shared->msgs[idx].ts, &tm);
    }
}

```

```

        strftime(timestr, sizeof(timestr), "%H:%M:%S", &tm);
        printf("[%s] %s> %s\n", timestr, shared->msgs[idx].user, shared->msgs[idx].text);
    }

    if (sem_post(mutex) == -1) perror("sem_post");
}

int main() {
    int is_creator = 0;
    init_shared(&is_creator);

    char user[MAX_USER];
    printf("Masukkan nama Anda: ");
    fgets(user, sizeof(user), stdin);
    user[strcspn(user, "\n")] = '\0';

    int pilih;
    char pesan[MAX_TEXT];
    while (1) {
        printf("\n1. Kirim Pesan\n2. Lihat History\n3. Keluar\nPilih: ");
        scanf("%d", &pilih);
        getchar();

        if (pilih == 1) {
            printf("Pesan: ");
            fgets(pesan, sizeof(pesan), stdin);
            pesan[strcspn(pesan, "\n")] = '\0';
            kirim_pesan(user, pesan);
        } else if (pilih == 2) {
            lihat_history();
        } else if (pilih == 3) {
            break;
        }
    }

    if (shared) munmap(shared, sizeof(SharedData));
    if (mutex) sem_close(mutex);
}

```

```

    if (shared) munmap(shared, sizeof(SharedData));
    if (mutex) sem_close(mutex);
    return 0;
}

```

```
root@RIDHO:~# gcc chat_aplikasi.c -o chat_aplikasi -lrt -pthread
root@RIDHO:~# ./chat_aplikasi
Masukkan nama Anda: Ridho

1. Kirim Pesan
2. Lihat History
3. Keluar
Pilih: 1
Pesan: Halo semua!!!

1. Kirim Pesan
2. Lihat History
3. Keluar
Pilih: 1
Pesan: Bagaimana kabarmu?

1. Kirim Pesan
2. Lihat History
3. Keluar
Pilih: 2
[19:41:04] Ridho> Halo semua!!!
[19:41:34] Putra> Halo Ridho
[19:42:27] Ridho> Bagaimana kabarmu?
[19:42:42] Putra> Baik, terima kasih

1. Kirim Pesan
2. Lihat History
3. Keluar
Pilih: 3
root@RIDHO:~# |
```

```

root@RIDHO:~# ./chat_aplikasi
Masukkan nama Anda: Putra

1. Kirim Pesan
2. Lihat History
3. Keluar
Pilih: 1
Pesan: Halo Ridho

1. Kirim Pesan
2. Lihat History
3. Keluar
Pilih: 2
[19:41:04] Ridho> Halo semua!!!
[19:41:34] Putra> Halo Ridho

1. Kirim Pesan
2. Lihat History
3. Keluar
Pilih: 1
Pesan: Baik, terima kasih

1. Kirim Pesan
2. Lihat History
3. Keluar
Pilih: 2
[19:41:04] Ridho> Halo semua!!!
[19:41:34] Putra> Halo Ridho
[19:42:27] Ridho> Bagaimana kabarmu?
[19:42:42] Putra> Baik, terima kasih

1. Kirim Pesan
2. Lihat History
3. Keluar
Pilih: 3
root@RIDHO:~# |

```

#### 4. Kesimpulan

Dari praktikum ini dapat disimpulkan bahwa:

- IPC memungkinkan komunikasi antar proses berjalan efisien.
- Shared Memory sangat cepat karena tidak melalui kernel secara langsung.
- Semaphore penting untuk menjaga konsistensi data bersama.
- Implementasi chat sederhana ini menjadi contoh nyata bagaimana proses bisa berinteraksi secara konkuren di sistem operasi.